

1. Define data and task parallelism.

Data parallelism distributes subsets of the same data across different computing cores and performs the same operation on each core.

Task parallelism distributes not data but tasks across multiple cores. Each task is running a unique operation.

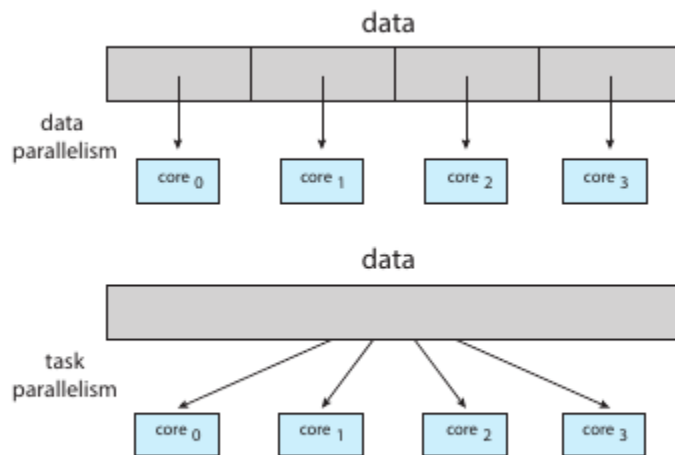


Figure 4.5 Data and task parallelism.

2. Two Differences Between User-Level Threads and Kernel-Level Threads

User-Level Threads	Kernel-Level Threads
Managed in user space by thread library	Managed directly by the operating system kernel
Faster to create and manage	Slower because system calls are required
Kernel is unaware of user threads	Kernel fully manages and schedules threads
One blocking call blocks the entire process	Blocking affects only the concerned thread

Summary : User-level threads are faster and efficient but have limited functionality, whereas kernel-level threads provide better multitasking and parallel execution.

3. Resources Used When a Thread is Created

When a thread is created, it uses:

- Thread ID
- Register set
- Stack
- Program counter

Threads share the following resources with other threads of the same process:

- Code section
 - Data section
 - Open files
 - Heap memory
-

Difference from Process Creation

When a process is created, the operating system must allocate:

- Separate address space
- Memory resources
- File descriptors
- Process Control Block (PCB)

Process creation requires more system resources and time because each process has its own separate memory and resources.

On the other hand, thread creation is faster and more efficient because threads share most resources of the parent process.

4. Explain Amdahl's Law. Calculate the speedup gain of an application that has a 60 percent parallel component for two processing cores and four processing cores.

5. Is it possible to have concurrency but not parallelism? Explain.

Answer:

Yes, it is possible to have concurrency without parallelism.

Concurrency means multiple processes or threads are making progress during the same time period, but not necessarily running at the exact same moment. The CPU switches quickly between tasks using scheduling.

Parallelism means multiple tasks are running simultaneously, which requires multiple processing cores.

So, on a single-core processor, concurrency is possible through task switching, but true parallelism is not possible.

Example of Concurrency (without Parallelism)

Suppose a single-core CPU is running:

- a music player
- a web browser

The CPU quickly switches between these two tasks.

Both programs appear to run together, but actually only one runs at a time.

✓ This is **concurrency**.

Example of Parallelism

Suppose a dual-core CPU is running:

- a video editing program on Core 1
- a game on Core 2

Both tasks run at the exact same time on different cores.

✓ This is **parallelism**.

6. Describe about Java Threads. Illustrate different approaches to implicit threading, including threadpools, fork-join, and Grand Central Dispatch.

Java provides built-in support for multithreading through the Java Virtual Machine (JVM).

A Java thread is a lightweight unit of execution that allows multiple tasks to run concurrently within a program. The JVM runs on top of a host operating system such as Windows, Linux, or macOS. The JVM hides operating-system details and provides a common environment for Java programs.

The JVM specification does not define how Java threads map to operating-system threads. Different systems use different implementations:

- Windows uses the one-to-one threading model.
- Linux and macOS commonly use the Pthreads API.

Implicit threading transfers thread creation and management from programmers to the compiler or run-time library. Developers only define tasks that can run in parallel.

Advantages

- Simplifies concurrent programming
- Reduces programmer workload
- Improves performance on multicore systems

Approaches of Implicit Threading

1. Thread Pools

A group of threads is created in advance and reused for different tasks.

Benefits:

- Faster execution
- Limits number of threads
- Better CPU and memory usage

Example methods in Java:

- `newSingleThreadExecutor()`
 - `newFixedThreadPool()`
 - `newCachedThreadPool()`
-

2. Fork-Join Model

A large task is divided into smaller subtasks, executed in parallel, and then combined.

Used in divide-and-conquer algorithms like:

- Quicksort
- Mergesort

Java uses:

- `ForkJoinPool`
 - `RecursiveTask`
-

3. Grand Central Dispatch (GCD)

GCD is developed by Apple for macOS and iOS.

It automatically manages threads using:

- Serial queues
- Concurrent queues

Provides efficient parallel execution on multicore systems.